

Звіт
з лабораторної роботи №7
Стегня Віктора ПМі-44

Тема

Реалізація алгоритму Мерліна-Сігала для побудови всіх найкоротших шляхів до ініціатора в симуляторі розподіленої системи.

Опис реалізації

Реалізація виконана на базі вже наявного симулятора в папці lab7. Алгоритм Мерліна-Сігала інтегровано в модель мережі та локальну логіку вузлів так, щоб симуляція сама запускала обчислення, проходила всі раунди та коректно завершувалась після глобального оголошення результату.

У реалізації були додані такі ключові частини:

- побудова початкового кістякового дерева з ініціатором у корені;
- раундовий обмін повідомленнями з поточними відстанями між сусідами;
- оновлення значень `dist` і `parent` наприкінці кожного раунду;
- автоматичне завершення після $N - 1$ раундів, де N — кількість вузлів у мережі;
- журналювання подій та автоматична перевірка отриманих маршрутів відносно еталонного розрахунку.

Зміни в симуляторі

Файл	Зміни
<code>scheduler/implementation/node.py</code>	Реалізовано логіку побудови початкового дерева, обробку раундів алгоритму Мерліна-Сігала, оновлення найкращих відстаней, зміну батьківського вузла та фінальне оголошення результату.
<code>scheduler/implementation/current_network.py</code>	Задано фіксовану вагову неорієнтовану мережу з 6 вузлів, ініціалізацію стартового

	повідомлення, автоматичну перевірку результатів і формування файлів підсумку.
scheduler/core/observer.py	Додано коректне завершення симуляції після повного завершення мережі без зависання в нескінченному циклі.
scheduler/abstract/abstract_network.py	Додано метод <code>is_finished()</code> , який використовується як базова точка контролю завершення.
scheduler/settings/network_settings.py	Налаштовано детермінований режим запуску без випадкових зовнішніх запитів, а також шляхи до файлів журналу й підсумкових результатів.

Хід роботи алгоритму

Алгоритм виконується в дві основні фази.

- На першій фазі будується початкове кістякове дерево. Ініціатор розсилає повідомлення `TREE_DISCOVER`, кожен вузол, який отримав таке повідомлення вперше, фіксує батька й продовжує поширення по ще не відвіданих сусідах. Після обходу піддерева вузол надсилає `TREE_ACK` назад до батька.
- На другій фазі ініціатор запускає раунди алгоритму Мерліна-Сігала. У кожному раунді вузол порівнює всі кандидатні відстані, отримані від сусідів, зі своїм поточним значенням `dist`. Якщо знайдено коротший шлях, вузол запам'ятовує нового потенційного батька.
- Після отримання повідомлень від усіх сусідів у поточному раунді вузол переходить до наступного раунду. Оскільки в мережі 6 вузлів, виконується 5 раундів.
- Після завершення останнього раунду ініціатор розсилає повідомлення `FINAL`, а всі вузли фіксують підсумкові значення у файлах звіту.

Таким чином, після раунду $N - 1$ кожен вузол має остаточні значення відстані до ініціатора та оптимального батьківського вузла, через якого проходить найкоротший маршрут.

Результати роботи

Симуляцію було перевірено запуском команди `python3 run_simulation.py` у папці `DistributedSystems/lab7`. Після виконання формуються файли:

- `test_results/lab7_events.txt` — детальний журнал подій усіх вузлів;
- `test_results/lab7_summary.txt` — фінальні значення відстаней і батьківських вузлів;
- `test_results/lab7_verification.txt` — автоматична перевірка результатів відносно еталонного алгоритму Дейкстри.

Підсумкова таблиця результатів має вигляд:

Вузол	Очікувана відстань	Отримана відстань	Фінальний parent	Початковий parent	Перевірка
1	0	0	—	—	ОК
2	7	7	1	1	ОК
3	9	9	1	1	ОК
4	20	20	3	2	ОК
5	20	20	6	6	ОК
6	11	11	3	1	ОК

З таблиці видно, що всі отримані відстані співпали з еталонними значеннями. Особливо показовими є вузли 4 і 6: у початковому дереві вони мали інші батьківські вершини, але після проходження раундів алгоритм коректно перебудував маршрути на коротші.

Висновки

У результаті виконання лабораторної роботи було реалізовано алгоритм Мерліна-Сігала для знаходження найкоротших шляхів до ініціатора у ваговій розподіленій мережі. Реалізація підтвердила теоретичну властивість алгоритму: після завершення $N - 1$ раундів усі вузли отримують остаточні значення відстаней та оптимальних батьківських вузлів.

Побудована симуляція показала, що поєднання початкового кістякового дерева з подальшим раундовим оновленням відстаней дозволяє коректно перейти від початкового, не обов'язково оптимального дерева, до дерева найкоротших шляхів. Автоматична перевірка підтвердила повний збіг результатів з еталонним розрахунком.